

Ask Inc42 — August release brief (frontend)

One release: streaming + the backend fixes + the new lanes. This supersedes [BRIEF-ASHISH-LANE-UX-CONTRACT.md](#) (kept for the detailed event notes).

Everything below is either already live on the backend or live on dev and ready for you to build against. **Six things need frontend work — items 2 to 6 are the ones that can break on launch day.**

0. Already live in production — nothing for you to do

Shipped and verified 2026-08-02, no contract change, same response JSON:

- **Cross-session cache fix.** Follow-up fragments were sharing one global cache entry, so a user asking "financials of company" could receive *another user's* answer. Session-scoped now.
- **Session memory fix.** Pronoun follow-ups ("who invested in them?") now resolve reliably — 100% entity resolution, up from a known intermittent failure.

Response JSON is **unchanged**: the same 19 fields ([output](#), [sources](#), [entity_links](#), [suggestions](#), [mode](#), [tool_count](#), ...) whichever internal path answers. Verified both ways.

1. Streaming — you've tested this. Ship it.

Stream slot [master-agent-v2-stream \(9I4y0BfNmEEFfqfe\)](#) is exactly as you tested: agent tV2.2, 36 nodes, [activity_plan](#) + native token streaming. We have deliberately **not** touched it. Ship, then let it soak before anything else lands under it.

2. The three lanes — auto-routed, never picked by the user

Queries are routed automatically. **There is no mode picker.** The user sees which lane answered and can ask for more afterwards.

Public label	Subtitle	Typical
Answer	Seconds	~3.5s
Analysis	Under a minute	~20s
Research	A few minutes	~45s

Internal names (Snap / Scan / Deep) appear in our logs and code — **never show them to a user.**

Events, in order

activity_plan — already live, arrives ~1.5s, before any tokens:

JSON

```
{ "type": "activity_plan", "query_type": "people", "mode": "fast",
  "steps": [ "Searching people & founders", "Linking companies", "Synthesizing" ] }
```

Render the steps as a checklist immediately. **Don't use mode for the badge** — see below.

lane —  live on dev, arrives once routing is decided:

JSON

```
{ "type": "lane", "lane": "answer", "label": "Answer", "eta_seconds": 4 }
```

lane ∈ **answer** | **analysis** | **research**. Render the badge with the ETA: *"Research — screening 40+ companies, ~45s"*.

activity_plan fires *before* the router has decided, which is why it can't name the lane and **lane** is a separate event.

Why the ETA matters: P90 is 52.6s. Silence at 50s reads as a hang. A named lane with a realistic ETA turns waiting into *justified* waiting. The ETAs are measured medians, not aspirations.

final —  live on dev (added 2026-08-11), the **last** event before the stream closes, on **every** path (agent, fast-path, cached, validation error):

JSON

```
{ "type": "final", "payload": { "output": "...full answer markdown...", "sources": [...],  
  "entity_links": [...], "suggestions": [...], "mode": "fast", "session_id": "...",  
  "_elapsed_ms": 16000, "_fallback": null } }
```

`payload` is exactly what the non-streaming slots return as their response body. **Treat `payload.output` as the authoritative answer text** — render tokens as they stream for liveliness, then reconcile against `payload.output` when `final` arrives (they can differ in whitespace/formatting; `output` wins). Sources, entity links and follow-up suggestions only exist here — there is no other way to get them from a stream.

Two shapes to handle:

- **Cached answers** (same session re-asking): the stream is **only `final`** — zero tokens, closes in <1s. `payload.cache_hit` is `true`.
- **Validation/rate-limit errors**: also `final`-only; `payload.error` is set and `payload.output` is absent. Render the error, not an empty answer.

3. Escalation — a second `lane` event, no new event type

If the fast path finds thin data it promotes to the full agent mid-flight. You'll receive a **second `lane` event with a higher lane** — e.g. `answer` then `analysis`.

Treat any second `lane` event as an escalation: *"Thin data on the fast path — going deeper (+25s)"*. **Append** to the checklist; don't reset it.

Verified on dev: `activity_plan` → `lane:answer` → `lane:analysis`.

4. Render on stream close — never gate on having seen a token

This is the most likely launch-day breakage.

Two cases produce a complete, correct answer with **no token stream at all**:

1. **Fast-path answers** never run the agent, so nothing streams — you get `activity_plan`, then `lane:answer`, then the final response.

2. **Escalated answers** sometimes stream nothing either. Measured, pre-existing, and non-deterministic: two runs of the same query gave **81 stream events with 0 characters** and **95 events with 1,885 characters**. Roughly 50/50. (Cause not yet established; the final response is always correct.)

So: render the final response whenever the HTTP stream closes, regardless of whether any text arrived. A client that waits for a first token will hang on our *fastest* queries.

Update 2026-08-11 — this is now mechanical, not defensive. The `final` event (section 2) arrives on every path including the zero-token ones, carrying the full answer in `payload.output`. Implement `final` and both no-token cases above are covered by construction: render `payload.output` when it arrives, done. This *answers question 1 in section 7* — the client doesn't need to handle "stream closed with no answer" as a special case anymore, it needs to handle the `final` event.

5. "Go deeper" — after the answer, never before backend live on dev

Put a **Go deeper** button on completed `answer` and `analysis` responses. It re-submits the same query with:

JSON

```
{"chatInput": "<same query>", "session_id": "<same session>", "force_lane": "analysis"}
```

`force_lane` $\in$ `analysis` | `research`. Verified end to end — the same query returns `lane:answer` (auto), `lane:analysis` or `lane:research` (forced), ETAs 4 / 20 / 45s.

Please log every click. Each one is a labelled routing error and it's the only telemetry that will tell us whether auto-routing is under-selecting depth.

There is deliberately **no upfront picker**: 62% of our multi-turn sessions span more than one complexity level, so a sticky mode would need changing mid-conversation; and industry data shows under 5% of users ever change a default while routing *raises* depth usage.

6. ⚠️ Agent V3 — change this now, it's backward compatible

V3 is coming (not urgent; the forcing function is the **Dec 11** gpt-5.4 deprecation). It **changes the stream shape**:

	agent begin	agent end	text events
V2 (what you tested)	1	1	15
V3	3	3	29

V3 re-enters the agent per tool iteration, emitting a begin/end pair each time. **A client that finalises on the first **end** will truncate answers to the first fragment.**

Fix: treat **end** as "*this segment finished*", accumulate across cycles, finalise on stream close. **This behaves identically on V2**, so please land it now rather than racing the V3 deploy.

7. Two questions we need answered

1. **Does the client render a final response that arrives with no preceding token stream?** (items 4 — both the fast-path and escalated cases) → **Resolved by design 2026-08-11**: implement the **final** event (section 2) and this is covered by construction. The question becomes: *confirm you'll implement **final**.*
 2. **Which webhook will the live frontend call after your deploy** — **master-agent-v1** or **master-agent-v2-stream**? This decides when the fast lane actually reaches users.
-

8. Sequence

1. **You ship streaming** against the stream slot as-is. Soak a few days.
2. We add the fast lane + lane events to the stream slot — gated on your answer to Q1.
3. Agent V3. **Only the streaming slot is gated on item 6.** The non-streaming slots (**master-agent-v1**, staging, GPT twin) return plain JSON and emit no **begin/end** events at all, so V3 there is invisible to any client and can ship independently. V3 on the *stream* slot waits for item 6 to be in your client.
4. Lane badges / "Go deeper" can land with 2 or after; they degrade gracefully (an unknown event type is ignorable).

Nothing here changes the response JSON, so each step is independently revertible.